01 / ROLE AND ROADMAP

Nakul's Phase 1 work guide

PROJECT 11 / AUTOMATIC CLOTHING CATEGORY RECOGNITION

Inventory Image Tagging via Handcrafted Features and a Small CNN
Phase 1 focus: classical machine learning on Fashion-MNIST.

Your role: Project integrator and research lead

Create the repository and protocol before anyone trains a model.

The five-person division

Member	Primary ownership
Nakul	Integration, literature and report Repository, master notebook, report assembly
Sudeshhna	Data quality and EDA Loader, fixed split, audit, EDA figures
Puneet	Features, Logistic Regression and kNN Pixel/HOG features and four model rows
Samarth	Random Forest, SVM and run harness Four model rows and experiment logs
Neelanshu	Evaluation and error analysis Metrics, timing, complexity, error figures

How to use this guide

Read pages 2-4 first, then work through your practical pages in order. Pages 13-16 explain handoffs, viva preparation, the proposed schedule and sources. Your guide is a work plan and learning manual, not a completed experiment or a submission-ready report.

All five members must understand the entire pipeline. Ownership means you implement and explain an area; it does not mean the others can skip learning it. Each member also writes their report section, prepares their slides, and reviews another member's work.

Prepared 27 October 2026. All workload estimates, code organization, hyperparameter grids and internal milestones are recommendations, not faculty rules. No model scores have been invented.

02 / SHARED FOUNDATIONS

What Phase 1 actually requires

The supplied CSAI instructions are the course requirements. Your project description supplies the technical problem. This guide adds a practical way to satisfy both; it does not turn its suggestions into extra faculty requirements.

Course requirement	Evidence the team should prepare
Literature review	A cited comparison of relevant sources and the gap your experiments investigate.
Dataset quality and EDA	Shape/range/label checks, class counts, sample images and a written audit.
Feature engineering	Normalized flattened pixels and a justified HOG configuration.
Theoretical rigor	Explain each model, its assumptions, tuning choices and limitations.
Model application	Two representations with Logistic Regression, kNN, Random Forest and SVM.
GitHub and code quality	Reproducible code, clear README, saved settings, outputs and contribution evidence.
Presentation and individual viva	Every member presents their work and explains the shared experiment.
Project report	A coherent report in LaTeX/IEEE format, using the course template if supplied.

Scope boundary

Complete the classical ML component now. Save the CNN, pooling, dropout, CNN early stopping, AML-versus-CNN experiments and any implemented hybrid approach for Phase 2. A brief future-work paragraph is enough at this stage. The Phase 1 comparison is pixels versus HOG across four classical classifiers.

The attachment gives no Phase 1 evaluation date, page limit, presentation duration, grading weights, minimum accuracy, or required number of papers. Its September dates concern project/team selection. Check the course channel for later evaluation announcements; do not treat the proposed schedule here as an official deadline.

03 / SHARED FOUNDATIONS

Machine learning from zero

Here, **AML** means the classical/applied machine-learning part described in your project. You do not hand-write a rule saying "sleeves mean shirt." Instead, you show a model labeled examples so it can learn a rule from image measurements.

Term	Plain meaning in this project
Sample; label	One clothing image; its correct category number. The label is the answer.
Feature; representation	A numeric input. One image becomes either 784 pixel values or a HOG vector.
X and y	X is the feature matrix: one row per image. y is the matching list of labels.
Model; fit; predict	A learned decision rule; learn from X and y; assign labels to new X.
Parameter	Something learned during fitting, such as a Logistic Regression coefficient.
Hyperparameter	A setting chosen before fitting, such as k in kNN or C in an SVM.
Training / validation / test	Practice examples / development exam / final independent exam.
Overfitting	Doing very well on training data but failing to generalize to held-out data.
EDA	Exploratory data analysis: inspect the data before deciding how to model it.
Reproducibility	Another person can repeat the process using the same data, code and settings.

Walk through a single image

A grayscale image is a small grid of brightness values. Divide its raw pixel values by 255 to put them on a 0-1 scale. Flatten the grid to a vector, or compute a HOG description of local edge directions. A fitted classifier then maps that vector to one of the category IDs. Map the ID to the correct name for display.

Your first learning exercise

Before your assigned work, use a notebook to print an array's shape, show one image, print its label, and call fit then predict on a tiny example. Explain aloud why a row of X must stay attached to the same row of y. If that connection is broken, model training becomes meaningless.

Important nuance: flattening does not erase pixel values or their recoverable ordering. It gives a conventional classifier no built-in local-neighborhood or translation structure. HOG encodes local geometry, but its feature rule is fixed before classifier training.

04 / SHARED FOUNDATIONS

One shared, fair experiment

Freeze this protocol before comparing models

- Keep the official training and test partitions separate. Create a stratified 80/20 split of the official training partition with random_state=42: 48,000 fit images and 12,000 validation images. Save the indices once; everyone reuses them.
- Use two fixed representations and four model families: pixels/HOG × Logistic Regression/kNN/Random Forest/SVM. This produces eight main comparison rows. Add one DummyClassifier reference row outside those eight.
- Train on the fit partition, choose settings using validation macro F1, and use validation accuracy as a tie-breaker. If still tied, choose the simpler or faster option using validation measurements. Log the rule before seeing test scores.
- Fit learned scalers only on fit data. A fixed divide-by-255 transform and fixed per-image HOG extraction do not learn dataset statistics. Use a Pipeline for StandardScaler plus the classifier.
- After freezing one configuration per row and naming the validation-selected overall model, evaluate the eight frozen models on the official test partition. Do not choose new hyperparameters or a new overall winner from test results.

Training size and compute policy

Default: all eight final models use the same 48,000-image fit partition and the same 12,000-image validation partition. Keep those fitted models for final testing; do not silently refit some models on all 60,000 training images. This preserves the same split for Phase 2.

If compute makes this infeasible, agree before final runs on one saved stratified 12,000-image subset drawn only from the 48,000 fit images. Use that identical subset for all eight rows, retain the same validation/test partitions, and label the table "12,000 fit images per model." Exploratory subset runs must never masquerade as full-size results.

Required result package

Report accuracy, macro F1, recall for all ten classes, training time, inference time with a stated protocol, and a meaningful model-complexity description. Explain when a classical model has no neural-network-style parameter count. Save all eight rows, including weak runs; discuss shirt/T-shirt/pullover confusion using observed examples.

Protocol choices on this page are recommendations for consistency. The official split and the stated project metrics come from your project description; the specific validation fraction, seed and selection rule are proposed here.

05 / YOUR WORK

Your ownership and first session

You are the integrator, not the person expected to complete everyone's unfinished work. Your technical job is to make the independently written parts form one reproducible pipeline. Your research job is to connect the question, methods and evidence into a coherent argument.

What you personally build

- **Repository and protocol:** a readable folder structure, one configuration file, an agreed run schema and a README with reproducible steps.
- **Master notebook:** notebooks/00_phase1_master.ipynb, which calls the loader, features, model runners and evaluator in the correct order. Reuse functions rather than copy-pasting four private notebooks.
- **Research synthesis:** docs/literature_matrix.csv and a related-work section comparing the sources the team actually read.
- **Report and presentation:** report/main.tex, references.bib, merged figures, contribution statement and a coherent slide narrative. Other members supply their own technical sections.

First 90 minutes

0-20 min: explain the Phase 1 boundary and learn X, y, fit, predict and validation. **20-40 min:** create the repository, invite the four teammates and add the protocol. **40-65 min:** make the environment import NumPy, scikit-learn and scikit-image. **65-90 min:** agree the sample-ID convention and run a tiny classifier with synthetic data to learn the interface.

Agree these decisions once

Use Python and scikit-learn for classical models, scikit-image for HOG, matplotlib for plots and torchvision only as a data loader if convenient. A CPU environment is sufficient to begin. Record actual installed versions. Choose one owner for final execution so laptop differences do not distort timing comparisons.

Your handoff is accepted when

A teammate can clone the repository, follow the README, obtain the same saved split, run one smoke experiment and locate its validation predictions and metrics. The master notebook must explain why each stage exists, not merely execute hidden functions.

Suggested effort: 20-26 hours including integration, literature, report assembly and rehearsal. Every teammate writes their own material, so final editing remains manageable.

06 / YOUR WORK

Build a repository beginners can use

A repository is the shared project folder tracked by Git. A commit is a saved change with a message. A branch lets someone work without immediately changing the shared main branch. Review a teammate's change before merging it.

```
clothing-phase1/
  README.md
  requirements.txt
  config/phase1.json
  src/data.py
  src/features.py
  src/models.py
  src/experiments.py
  src/evaluate.py
  notebooks/00_phase1_master.ipynb
  notebooks/01_data_eda.ipynb
  notebooks/02_features_lr_knn.ipynb
  notebooks/03_rf_svm.ipynb
  notebooks/04_evaluation.ipynb
  data/split_indices.npz
  artifacts/  results/  figures/
  report/main.tex  report/references.bib
  docs/protocol.md  docs/contributions.md
```

Minimal setup recipe

```
python -m venv .venv
# macOS/Linux:
source .venv/bin/activate
# Windows PowerShell instead:
# .venv\Scripts\Activate.ps1
python -m pip install numpy pandas scikit-learn scikit-image
python -m pip install matplotlib jupyter joblib torch torchvision
python -m pip freeze > requirements.txt
```

In a hosted notebook, install missing packages in that notebook environment and download the code there. Save results outside temporary notebook storage. A GPU is unnecessary for these classical models. If torch and torchvision fail to import together, use matching builds or the direct IDX loader described in Sudeshhna's guide.

Keep the repository useful

Ignore .venv/, raw dataset downloads, temporary caches and oversized binary artifacts. Keep the small split file, configs and result tables under version control. Store larger trained models in an agreed artifact location with instructions and hashes. Never commit passwords, tokens or personal notebook credentials. These are normal repository hygiene steps.

README sections: problem; Phase 1 scope; team; setup; dataset retrieval; exact run order; compute profile; result locations; report build; limitations; contributions; references. Pin the tested environment after a successful smoke run, not after unrelated packages accumulate.

07 / YOUR WORK

Integrate one end-to-end pipeline

Define the interfaces before modules grow

Sudeshhna returns raw images and integer labels, plus original IDs and saved indices. Puneet's transform takes raw uint8 images and returns rows of features in exactly the same order. Model owners return fitted Pipelines and run records. Neelanshu evaluates predictions keyed to sample IDs.

```
# Illustrative orchestration; adapt to team module names.
raw, labels = load_official_train()
idx = np.load("data/split_indices.npz")
tr, va = idx["train_idx"], idx["val_idx"]
Xtr = make_features(raw[tr], feature_name)
Xva = make_features(raw[va], feature_name)
pipe = make_model(model_name, settings)
pipe.fit(Xtr, labels[tr])
pred = pipe.predict(Xva)
save_predictions(run_id, "val", va, labels[va], pred)
# Test loading/evaluation is a separate, later stage.
```

The actual master notebook imports these functions from src/. It should not reimplement feature extraction or metric formulas. Saving the transformer settings with the model is essential: a fitted classifier alone cannot tell a future user whether its inputs were pixels or HOG.

Put decisions in one config

```
{
  "seed": 42,
  "validation_fraction": 0.2,
  "train_profile": "full_48000",
  "selection_metric": "validation_macro_f1",
  "hog": {"orientations": 9, "pixels_per_cell": [4, 4],
          "cells_per_block": [2, 2], "block_norm": "L2-Hys"},
  "final_refit": false,
  "test_status": "locked"
}
```

Your integration checkpoints

- First smoke run: use saved subsets of 2,000 fit and 1,000 validation images. It proves the plumbing works; label it smoke only. Confirm the final run uses the agreed final-size profile.
- Before accepting a module, verify shape, dtype, finite values and row order on a few IDs. Never infer correctness merely because the notebook displays a high score.
- Freeze configuration hashes and the validation-selected model ID before changing test_status. Final test artifacts live separately from tuning artifacts.

Do not assume a missing function is provided by scikit-learn: the names above are contracts your team must implement. Starter fragments in these guides are not a ready-made runnable repository.

08 / YOUR WORK

Lead a real literature review

A literature review explains what others did and how it motivates your comparison. It is not a list of links or five copied abstracts. Read the dataset paper [R2], a source for HOG [R3-R4 and its original paper], and relevant classical-model material [R6-R9]. Treat official documentation as implementation/theory support, not as a replacement for research papers.

Give everyone a reading contribution

Member	Reading and written contribution
Nakul	Synthesize the problem, dataset motivation and representation gap; edit the related-work narrative.
Sudeshhna	Read the Fashion-MNIST paper; describe the data and why it differs from a real catalogue.
Puneet	Read HOG material and LR/kNN documentation; explain representations and local similarity.
Samarth	Read SVM/RF theory and docs; explain margin-based and ensemble approaches.
Neelanshu	Read evaluation guidance; explain metrics, leakage prevention and comparison limits.

Use a comparison matrix

For each source add: citation key; full title/authors/year; task and dataset; feature representation; model; split and evaluation method; reported finding; limitation; what your experiment borrows or tests. Leave an uncertain field marked "not verified" until someone reads it. Do not invent scores or cite a claim you cannot locate.

Suggested related-work paragraph structure

Paragraph 1: clothing classification is a supervised visual recognition problem; Fashion-MNIST provides a controlled benchmark. **Paragraph 2:** raw intensities offer a simple reference while HOG encodes local edges. **Paragraph 3:** linear, neighbor, tree-ensemble and margin-based classifiers make different decisions from these features. **Paragraph 4:** your contribution is a controlled, reproducible comparison and class-specific error analysis, not a new algorithm.

Separate evidence from expectation

Before running: "We hypothesize that HOG will help some models distinguish clothing silhouettes." After running: state only what the recorded comparison supports. If HOG loses, explain the measured trade-off rather than rewriting the hypothesis as a guaranteed benefit. Do not present third-party leaderboard scores as your own baseline or an assured target.

Suggested target: 3-5 relevant research sources plus implementation documentation if time allows. This is a planning recommendation; the attachment specifies no minimum count.

09 / YOUR WORK

Theory you must connect across models

For theoretical rigor, explain what each method assumes about the input and how its hyperparameters control flexibility. You do not need to derive every optimization algorithm, but you must connect equations to the code your team used.

Method	Core idea	Choice to justify
Logistic Regression	Class scores are linear in the features; softmax turns scores into class probabilities.	C controls inverse regularization strength; scaling affects the penalty.
kNN	A new image receives a vote from nearby stored training feature vectors.	k, uniform/distance weighting, and what distance means in the chosen representation.
Random Forest	Many decision trees learn different partitions; their outputs are combined.	Tree count, depth, leaf size and randomized training reduce or control variance.
RBF SVM	A margin-based classifier uses a kernel to express nonlinear similarity.	C controls the error/margin trade-off; gamma controls the locality of the kernel.

Explain overfitting using your actual evidence

A flexible model can memorize training peculiarities. Compare training and validation results only when both were measured on clearly stated sample sizes. High training F1 with much lower validation F1 suggests overfitting, but it does not prove a specific cause. Underfitting can arise when the representation or model is too restrictive.

Explain fair comparison without pretending everything is identical

The same data split, sample IDs, metric definitions and final training size are essential. Preprocessing may differ sensibly: scaled inputs for LR/SVM, 0-1 pixels for kNN, and no learned scaler for RF. State these choices. Equal feature dimensions are not necessary; reporting them is. Different search budgets must be disclosed.

The representation gap

A flattened vector still contains the image intensities but carries no built-in spatial inductive bias for a conventional classifier. HOG explicitly aggregates local edge directions yet is not trained end to end for clothing labels. This supports asking whether learned spatial features might help in Phase 2. It does not establish that a CNN will win.

Complexity is not one universal number

A linear model has coefficients, kNN stores examples, a forest stores trees and an RBF SVM stores support vectors plus coefficients. Ask Neelanshu for structural counts and serialized bytes with labels. Do not rank their expressiveness from one misleading "parameter count" column.

10 / YOUR WORK

Assemble the LaTeX/IEEE report

Use the faculty template if one is provided. Otherwise start from a standard IEEE conference template in LaTeX, for example in Overleaf. The guide recommends structure; the attachment supplies no page limit. Every member writes a technically correct draft of their own work and checks it after editing.

Section	Owner; required content
Title, abstract, introduction	Nakul: problem, Phase 1 question, approach and actual main finding. Write the abstract last.
Related work	Nakul integrates all members' verified reading notes and citations.
Dataset and EDA	Sudeshhna: source, label map, audit, split, representative figures and limitations.
Features and methods	Puneet: pixels/HOG and LR/kNN. Samarth: RF/SVM and their theory/settings.
Experimental protocol	Samarth + Neelanshu: training size, selection rule, tuning budget, versions and timing method.
Results and discussion	Neelanshu + model owners: verified eight-row table, per-class recall, errors and trade-offs.
Conclusion and future work	Nakul: answer the Phase 1 question; reserve CNN comparison for Phase 2.
Contributions and references	All members verify real work, citations, figure provenance and artifact links.

Minimal LaTeX skeleton

```
\documentclass[conference]{IEEEtran}
\usepackage{graphicx}
\usepackage{booktabs}
\begin{document}
\title{Clothing Recognition with Pixels and HOG}
% Add real author/affiliation details here.
\maketitle
\begin{abstract}
% Write after measured results are finalized.
\end{abstract}
\section{Introduction}
\section{Dataset and Methods}
\section{Experiments and Results}
\section{Conclusion}
\bibliographystyle{IEEEtran}
\bibliography{references}
\end{document}
```

Before submission, compile and inspect every page. Fix broken references, unreadable plots, missing figure captions, inconsistent names and tables with different training sizes hidden in one ranking. No placeholder scores may remain. Source code for the report belongs in the repository.

11 / YOUR WORK

Presentation and reproducible demo

Build the presentation around one question: how do raw pixels and fixed local-edge features change classical clothing classification? Use measured evidence to answer it. Until the course gives a duration, plan a flexible 10-12-slide version and shorten it if needed.

Suggested slides	Lead; message
1-2: Problem and Phase 1 scope	Nakul: input/output, why catalogue tagging matters, research question and exclusions.
3-4: Data and EDA	Sudeshhna: representative classes, quality checks and the locked split.
5-6: Feature representations	Puneet: one image, flattened pixels, HOG visualization and dimensions.
7: Models and tuning	Samarth: four different decision rules, compact settings and compute policy.
8-10: Results and errors	Neelanshu: accuracy/F1 table, per-class recall and paired error examples.
11-12: Conclusion and next phase	Nakul: supported trade-offs, limitations and the future CNN question.

A demo that works on evaluation day

- Start from saved models and a small predetermined sample list. Show input image, feature option, predicted category and true category. Use a known error as well as a correct example.
- Run the metric calculation from a saved prediction CSV to show that the result table is derived from evidence. Avoid live retraining of a long SVM during a short presentation.
- Keep the master notebook in execution order. Label cached results honestly. Have a backup PDF of figures and the environment/setup notes if the network is unavailable.

Rehearse as a team

Ask each member to explain someone else's part at a basic level. Ask why a held-out split is needed, why HOG could help or hurt, and what one confusing pair teaches you. Swap questions instead of reciting a prepared script. The faculty assesses viva individually.

Your review partner

Samarth reviews your README on a fresh environment or notebook session. You review Sudeshhna's split contract and confirm that everyone uses it. Neelanshu checks all displayed result numbers against the saved tables. Any disagreement is resolved from artifacts rather than memory.

A polished interface or deployed website is optional and unnecessary for the stated Phase 1 evaluation. Prioritize the classifier notebook, evidence, report and understanding.

12 / YOUR WORK

Your final integration audit

Check the chain from question to evidence

- The title and abstract make clear this submission covers Phase 1 classical ML. No completed CNN or hybrid results are claimed.
- Every main comparison row uses the same agreed fit/validation/test membership and final training size. The report names the feature extraction and scaler for each row.
- All four algorithms appear on both features. The dummy reference is separate. Configurations and the overall selected model were recorded before final test access.
- Accuracy, macro F1 and all ten class recalls come from saved predictions. Timing includes a stated machine, batch size, warm-up, repetitions and preprocessing boundary.
- Classical-model complexity is described honestly. Fit time and inference time use different columns and explicit units.
- Every report table and figure has an artifact path or run ID. Weak models and failed runs are explained. Claims about clothing confusions refer to observed counts and examples.

Check reproducibility and authorship

Use a clean session to follow the README through the smoke path. Confirm imports, relative paths and required files. Check package versions and data retrieval instructions. Make sure no absolute path from one person's laptop is required. Check that the report source compiles and every figure exists.

The contributions file should state actual ownership and meaningful supporting work. An entry can say: "Nakul integrated the master notebook, fixed prediction-schema inconsistencies, synthesized related work and assembled the report; reviewed split handling." Link real commits after work happens. Never create fictitious commit IDs or backfill fake work.

Before you call Phase 1 complete

Have all five members sign off on their own report content and perform one short cross-topic viva round. Save a final manifest listing the repository commit, config hash, split hash, eight model IDs, selected model ID and report file. Confirm any submission instructions or new deadlines in the course channel.

Your concrete completion standard

The whole project can be explained and reproduced from the repository, the result tables are traceable, the report is compiled, and every teammate can explain both their contribution and the shared method.

13 / TEAM DELIVERY

Handoffs and shared file contract

Each person works on their own files, commits meaningful changes, and opens a review request or asks a teammate to review. Share generated evidence with file paths and the exact run ID; a screenshot in a group chat is insufficient.

Sender → receiver	What is handed over; acceptance check
Sudeshhna → everyone	Loader, labels, split_indices.npz, audit.md. Row counts and train/validation IDs agree.
Puneet → Samarth + Neelanshu	Feature functions, HOG settings, feature metadata. A sample ID yields the same vector everywhere.
Puneet + Samarth → Neelanshu	Fitted pipelines, validation predictions, run logs and feature metadata. Every prediction has a split and sample ID.
Neelanshu → Nakul + model owners	Verified tables, error figures, timing method and audit findings. Results trace back to saved predictions.
All four → Nakul	Their report sections, figure captions, references, contribution notes and slides.
Nakul → everyone	Integrated notebook, draft report and slides. Each member checks their section and practices the full demo.

Use these identifiers consistently

sample_id: original row number within the official partition, plus **source_split** = official_train or official_test. Train/validation IDs both refer to official_train. Never use the row number of a shuffled feature cache as the original sample ID.

Prediction CSV: run_id, stage, source_split, sample_id, y_true, y_pred. stage is val or test. Do not invent probabilities when a classifier only exposes decision scores. Preserve deterministic sample order.

Run record: run_id, owner, git_commit, seed, split_file_hash, train_n, val_n, feature_name, feature_dim, feature_config, scaler, estimator, hyperparameters, package_versions, fit_seconds, status, warnings and artifact_paths. Keep validation and test metrics in separate fields/files.

Results CSV: run_id, stage, train_n, feature, model, accuracy, macro_f1, fit_seconds, predict_ms_per_image, end_to_end_ms_per_image, serialized_bytes, complexity_type, complexity_value. Put per-class recall in a separate ten-row table per run. Use seconds, milliseconds and bytes explicitly.

Every handoff includes: what changed, how to run it, expected shapes, the evidence checked, and known limitations. The person receiving it must rerun a small example before accepting it.

14 / TEAM DELIVERY

Individual viva practice

Practice without reading the answers. Explain each idea simply, then open the relevant code or figure as evidence. These are suggested talking points, not answers to memorize word for word.

Everyone must answer these

What is the difference between validation and test?
Validation guides development. Test measures the frozen procedure on held-out examples. Looking at test failures and then tuning would contaminate that final assessment.

Why use both pixels and HOG?
They test whether a fixed description of local edges helps compared with raw intensities. HOG may help some classifiers and hurt others; the experiment must establish the result.

Why is accuracy alone insufficient?
It summarizes total correctness and can hide a weak category. Macro F1 averages class F1 values equally; per-class recall shows which categories are missed.

What can you honestly conclude about a CNN now?
Only that a CNN is a motivated Phase 2 direction. There is no evidence that your future CNN outperforms your ML baseline until that comparison is run.

Nakul: questions about your own work

How do you know the project is reproducible?
A teammate can follow the README in a clean session and recover the shared split, run metadata and saved predictions; the master notebook imports the same reviewed modules.

What is your contribution beyond coordination?
Explain the orchestration code, interface checks, research synthesis and report integration you actually completed. Point to real files and commits.

Why is this a controlled comparison?
The models share data membership, training size and evaluation rules. Representation, algorithm and documented preprocessing choices are the intended differences.

Five-minute live demonstration

Show your own file and explain one nontrivial choice. Run a small cell or load a saved artifact. Trace one image from its original ID through features to a prediction. Read a validation metric correctly. Explain one failure, one limitation and one improvement reserved for Phase 2. Everyone must also describe the eight-row experiment and the shared split.

15 / TEAM DELIVERY

Suggested 10-day work plan

This is a relative plan, not an official deadline. Budget roughly 18-26 focused hours per person plus unattended compute; beginners may need longer. If you have fewer days, reduce the optional search grid and parallelize member-owned work. Keep data integrity, all eight baseline rows, and honest reporting.

Day	Your focus	Team gate
1	Create repo; agree protocol	All five understand scope and vocabulary
2	Read sources; problem statement	Environment works; literature notes started
3	Review data and split contracts	Data/split/feature contracts agreed
4	Run first integrated smoke demo	One tiny run reaches the evaluator
5	Document theoretical comparisons	Protocol and compute budget fixed
6	Merge feature/model modules	Eight comparable rows progressing
7	Check full run completeness	Configurations frozen; test run authorized internally
8	Integrate verified report results	Verified figures and individual text ready
9	Compile report; rehearse demo	One full rehearsal; report compiles
10	Final audit and submission package	Fresh-run check; contribution log complete

Definition of done for every member

- Your files run in the agreed environment, your inputs and outputs are documented, and a teammate has reviewed one small run.
- Your report section explains what you did, why you did it, what evidence supports it, and what its limits are. Captions and citations are included.
- Your contribution log names real files, commits, experiments and reviews. It records actual work, not an artificial equal number of commits.
- You can explain your own implementation and the shared project during an individual viva. You have practiced a backup demo using saved outputs.

If a run fails, preserve the error and settings. Tell the team early and revise the compute plan consistently. Do not fabricate results, omit failed rows without explanation, or quietly change the split.

16 / TEAM DELIVERY

Sources and how to use them

Course source: CSAI_Project_Instructions_Phase_1_and_Phase_2.md, supplied by you. Its Phase 1 list and individual-viva rule govern the required work. Your pasted Project 11 description supplies the feature/model comparison and metrics. Neither source contains actual results for your team.

Technical references below were checked on 27 October 2026. Stable documentation can change; record the versions actually installed for your experiments. Reference IDs such as [R3] in this guide point to these links. Read and cite original material in your submitted report; this guide is not a substitute for a literature review.

[R1] Fashion-MNIST official repository: data files, label map and official partitions

[R2] Xiao, Rasul and Vollgraf (2017), Fashion-MNIST dataset paper

[R3] scikit-image: HOG API and settings

[R4] scikit-image: HOG worked explanation

[R5] scikit-learn: leakage and preprocessing pitfalls

[R6] scikit-learn: LogisticRegression

[R7] scikit-learn: KNeighborsClassifier

[R8] scikit-learn: RandomForestClassifier

[R9] scikit-learn: SVC

[R10] scikit-learn: model evaluation and metrics

[R11] scikit-learn: stratified train_test_split

[R12] Torchvision: FashionMNIST loader

[R13] Dalal and Triggs (2005): original HOG paper, CVPR

[R14] scikit-learn: LinearSVC alternative and multiclass behavior

[R15] LeCun et al. (1998): author-hosted document-recognition paper record

Suggested research reading beyond API pages

Read Dalal and Triggs (2005), "Histograms of Oriented Gradients for Human Detection" [R13], for the original descriptor. Its task is human detection, so do not transfer its scores to clothing classification. LeCun et al. (1998), "Gradient-Based Learning Applied to Document Recognition" [R15], is useful Phase 2 background; it does not require a Phase 1 CNN implementation.

Citation discipline

For each source record title, authors, year, task, representation, model, evaluation protocol and relevance. Separate a paper's results from your results. Repository leaderboard entries are not automatically controlled comparisons or verified targets. Use actual bibliographic metadata in your LaTeX .bib file; never cite an unread paper as evidence for a specific finding.

Code blocks are starter recipes to adapt and run in your project environment. They have not been run on Fashion-MNIST as part of creating these guides. Toy calculations are illustrative, not measured project performance.